

AURA-HOI: Auditable Unified Representative Mining for High-Occupancy Itemsets

Minh Quan Van Ha
Independent Researcher

Abstract

High-occupancy itemset mining (HOIM) extends frequent itemset mining by requiring an itemset to occupy a sufficiently large fraction of the transactions in which it appears. This density-oriented objective is useful when support alone is not discriminative, but it also destroys the direct anti-monotonicity that makes classical frequent itemset mining efficient. Existing high-occupancy itemset miners further complicate evaluation because their outputs are not always the same object: some enumerate threshold-complete raw itemsets, while others report adaptive, maximal, top- k , or closed representative patterns. This paper presents AURA-HOI, an auditable high-occupancy itemset mining algorithm designed to separate scoring semantics from output views. AURA-HOI supports a raw fullset mode for direct comparison with HEP and DFHOI under a shared support-occupancy threshold, and a support-class representative mode for compact closed-output analysis. The method combines vertical bitset evidence, residual occupancy-envelope pruning, and a support-equivalence ledger. Experiments implemented in C and executed on a laptop-scale environment evaluate AURA-HOI, HEP, and DFHOI on five transactional datasets: `mushrooms`, `chess`, `retail`, `T10I4D100K`, and `kosarak`. Across all 35 dataset-threshold configurations, the three algorithms emit identical raw itemset counts under matched semantics. AURA-HOI is faster than HEP on 25 of 35 configurations and faster than DFHOI on 15 of 35 configurations; it also uses lower peak memory than HEP on 28 of 35 configurations and lower peak memory than DFHOI on 27 of 35 configurations. The results show that the proposed audit-oriented design preserves raw-output equivalence while providing competitive runtime and stable memory behavior across dense, sparse, synthetic, and large clickstream workloads.

Keywords: high-occupancy itemset mining; frequent itemset mining; closed itemsets; vertical bitsets; support equivalence; output normalization; occupancy pruning; reproducible pattern mining.

1 Introduction

Pattern mining studies the discovery of meaningful combinations of items from transactional data. In classical association-rule mining, the computational bottleneck is frequent itemset mining: rules are generated after identifying itemsets whose support exceeds a user-defined threshold [1, 2]. The tractability of frequent itemset mining is closely tied to the anti-monotonicity of support. If an itemset is infrequent, every superset of that itemset is also infrequent. This property enables level-wise pruning in Apriori [2], conditional database compression in FP-growth [4], and efficient transaction-id intersections in vertical approaches such as Eclat [9, 10].

Support, however, measures only how often an itemset appears. It does not measure how much of each supporting transaction the itemset explains. In long transactions, a small itemset may occur frequently while still covering only a small fraction of each transaction. High-occupancy itemset mining (HOIM) was introduced to address this limitation by using an occupancy score based on the ratio $|X|/|t|$ for itemset X inside supporting transaction t [3]. Deng’s original work proposed the HEP algorithm and upper-bound occupancy pruning for mining high-occupancy itemsets [3]. Nguyen et al. later proposed DFHOI and related algorithms based on equivalence classes and early pruning [7].

HOIM is more difficult to prune than support-only mining because occupancy is not anti-monotone under extension. Adding an item increases the numerator $|X|$, but it may shrink the support set. Therefore, an extension can increase or decrease occupancy relative to its prefix. This creates a need for safe upper bounds over descendants of a search prefix. It also creates a second evaluation problem: different HOI algorithms may emit different output views. A raw threshold-complete miner outputs every itemset satisfying the threshold predicate. A top- k miner outputs only the highest-ranked patterns. A maximal miner removes non-maximal patterns. A closed miner emits one support-equivalence representative per class. These outputs can all be useful, but they cannot be compared by raw count without normalization.

This paper proposes AURA-HOI (Auditable Unified Representative Algorithm for HOI), an HOI-only algorithm designed around one principle: every reported pattern must be traceable to a precise threshold predicate and to the support evidence that justifies it. AURA-HOI provides two main output views. The first is a raw fullset view, used for direct equality comparison with HEP and DFHOI. The second is a support-class representative view, where closed representatives are stored with support-bitset certificates. In the experiments, AURA-HOI is evaluated in raw summed-occupancy compatibility mode so that all compared algorithms solve the same task.

The contributions are as follows.

1. **Unified semantics.** The paper formalizes raw high-occupancy output, summed-occupancy compatibility, support closure, and support-class certificates in a single framework.
2. **Auditable algorithm design.** AURA-HOI uses vertical bitsets and support-class ledgers to make support evidence explicit and verifiable.
3. **Safe residual pruning.** A transaction-length-aware residual occupancy envelope upper-bounds the best possible descendant occupancy and enables no-false-negative pruning.
4. **Running example.** A complete five-transaction database is used throughout the paper to illustrate support, average occupancy, summed compatibility, closure, vertical bitsets, pruning, and the raw search behavior of AURA-HOI.
5. **Fair empirical protocol.** Experiments require matched raw-output semantics before interpreting runtime and memory comparisons.
6. **Laptop-scale reproducibility.** The reported results are produced by a C implementation on a commodity HP ProBook laptop with an Intel Core i7-8550U processor and 16GB RAM.

2 Related Works

2.1 Frequent itemset mining

Association-rule mining was introduced by Agrawal, Imieliński, and Swami as the discovery of implications between itemsets in large databases [1]. Apriori then established the canonical level-wise algorithm for association-rule mining by exploiting downward closure of support [2]. Subsequent methods improved the traversal and representation of transactional evidence. FP-growth avoids explicit candidate generation by compressing transactions into an FP-tree and recursively mining conditional pattern bases [4]. Eclat uses vertical transaction-id lists and computes support through intersections [9, 10]. These works form the algorithmic background of AURA-HOI because AURA-HOI also relies on vertical transaction evidence, but it uses that evidence for occupancy-aware auditing rather than support-only frequency.

2.2 Closed itemsets and support-equivalence classes

Frequent itemset mining can produce very large outputs. Closed itemset mining reduces redundancy by grouping all itemsets that share the same support set. Pasquier et al. introduced frequent

closed itemsets as a concise representation for association rules [8]. CHARM further developed efficient closed itemset mining with vertical tidset operations and closure properties [11]. The key idea is that support-equivalent itemsets can be represented by a closed itemset without losing support information. AURA-HOI transfers this idea into HOI mining, but does not confuse closed representatives with raw outputs. Instead, raw and closed outputs are treated as separate views over the same support evidence.

2.3 High-occupancy itemset mining

Deng introduced high-occupancy itemset mining to discover itemsets that occupy a sufficiently large proportion of the transactions in which they occur [3]. The original HEP algorithm uses occupancy lists and upper-bound pruning to cope with the non-anti-monotonicity of occupancy. Nguyen et al. proposed an efficient method based on equivalence classes and early pruning, including the DFHOI algorithm, to improve the efficiency of HOI mining [7]. Kim et al. investigated high-occupancy pattern mining in incremental databases, emphasizing that memory behavior matters in dynamic settings [5]. These works show that HOI mining is not merely a direct variant of frequent itemset mining; it requires its own scoring, pruning, and evaluation discipline.

2.4 Output semantics and fair comparison

A frequent source of ambiguity in pattern-mining evaluation is output mismatch. A threshold-complete miner, a closed miner, a maximal miner, and a top- k miner may all be correct under their own definitions while returning different counts. Therefore, a runtime advantage is scientifically meaningful only when the compared algorithms emit the same output view under the same threshold predicate. AURA-HOI is motivated by this problem. It keeps the raw fullset view for direct comparison with HEP and DFHOI, while preserving a closed support-class view for compact analysis.

3 Proposed Approach

3.1 Problem formulation

Let $I = \{1, \dots, m\}$ be an item universe and let $D = \{t_1, \dots, t_n\}$ be a multiset of transactions, where $t_q \subseteq I$ and $|t_q| > 0$. For itemset $X \subseteq I$, define the supporting transaction set and support count as

$$T(X) = \{q : X \subseteq t_q\}, \quad \text{supp}(X) = |T(X)|. \quad (1)$$

The exact average occupancy of X is

$$\text{aocc}(X) = \begin{cases} \frac{|X|}{\text{supp}(X)} \sum_{q \in T(X)} \frac{1}{|t_q|}, & \text{supp}(X) > 0, \\ 0, & \text{supp}(X) = 0. \end{cases} \quad (2)$$

Given minimum support $\sigma_{\min}$ and minimum average occupancy $\alpha \in (0, 1]$, the raw average-HOI set is

$$\mathcal{H}_{\text{raw}}(D, \sigma_{\min}, \alpha) = \{X \subseteq I : \text{supp}(X) \geq \sigma_{\min} \wedge \text{aocc}(X) \geq \alpha\}. \quad (3)$$

The public baseline behavior evaluated in this paper uses a shared threshold α as both a support threshold and a summed-occupancy threshold. Define the summed occupancy score as

$$\text{socc}(X) = |X| \sum_{q \in T(X)} \frac{1}{|t_q|}. \quad (4)$$

Under summed-occupancy compatibility, the benchmark predicate is

$$\text{supp}(X) \geq \lceil \alpha |D| \rceil \quad \text{and} \quad \text{socc}(X) \geq \alpha |D|. \quad (5)$$

All main experiments use Eq. (5). This ensures that AURA-HOI, HEP, and DFHOI are compared as raw fullset miners under the same repository-level semantics.

3.2 Running example

To make the definitions and algorithmic steps concrete, this paper uses one small transaction database as a running example. Let the item universe be

$$I_{\text{ex}} = \{A, B, C, D, E\},$$

and let the transaction database be

$$D_{\text{ex}} = \{t_1, t_2, t_3, t_4, t_5\}.$$

The transactions are shown in Table 1.

Table 1: Running example transaction database.

Transaction	Items	Length
t_1	$\{A, B, C, D\}$	4
t_2	$\{A, B, C\}$	3
t_3	$\{A, B, D\}$	3
t_4	$\{A, C, D, E\}$	4
t_5	$\{B, C, D, E\}$	4

This example is intentionally small but contains the main structural cases required by HOI mining. Some itemsets have high support but only moderate occupancy; some itemsets have lower support but cover a larger fraction of their supporting transactions; and some itemsets provide clear examples of support bitsets, closure, and branch pruning.

Consider the itemset $X = \{A, B\}$. It appears in t_1, t_2, t_3 , hence

$$T(\{A, B\}) = \{1, 2, 3\}, \quad \text{supp}(\{A, B\}) = 3.$$

Its exact average occupancy is

$$\text{aocc}(\{A, B\}) = \frac{|\{A, B\}|}{3} \left(\frac{1}{|t_1|} + \frac{1}{|t_2|} + \frac{1}{|t_3|} \right) = \frac{2}{3} \left(\frac{1}{4} + \frac{1}{3} + \frac{1}{3} \right) = 0.6111.$$

The summed occupancy score used by the repository-level compatibility mode is

$$\text{socc}(\{A, B\}) = 2 \left(\frac{1}{4} + \frac{1}{3} + \frac{1}{3} \right) = 1.8333.$$

If the shared threshold is $\alpha_{\text{ex}} = 0.35$, then the compatibility predicate uses

$$\sigma_{\min} = \lceil \alpha_{\text{ex}} |D_{\text{ex}}| \rceil = \lceil 0.35 \times 5 \rceil = 2$$

and

$$\xi = \alpha_{\text{ex}} |D_{\text{ex}}| = 0.35 \times 5 = 1.75.$$

Therefore, $\{A, B\}$ satisfies the support condition because $3 \geq 2$, and it also satisfies the summed-occupancy condition because $1.8333 \geq 1.75$. It is therefore a raw summed-compatible HOI in this running example. The same itemset also satisfies average occupancy at threshold 0.35 because $0.6111 \geq 0.35$.

The example also illustrates why average occupancy and summed-occupancy compatibility must be distinguished. If α_{ex} were increased to 0.40, then $\xi = 2.0$. The same itemset $\{A, B\}$ would still have average occupancy 0.6111, but it would fail the summed-compatible threshold because $1.8333 < 2.0$. Thus, the average predicate and repository compatibility predicate are related but not identical.

3.3 Output views and certificates

The support closure of itemset X is

$$\text{cl}(X) = \bigcap_{q \in T(X)} t_q. \quad (6)$$

An itemset C is support-closed if $\text{cl}(C) = C$. All itemsets with the same support bitset belong to a support-equivalence class. AURA-HOI stores a certificate for each closed representative:

$$\text{cert}(C) = (T(C), C, \mathcal{G}(C)), \quad (7)$$

where $T(C)$ is the support bitset, C is the representative, and $\mathcal{G}(C)$ denotes the generator paths or traversal intervals that reached the same support class. In implementation, a support class is first indexed by a hash and then verified by byte-level bitset equality. This avoids relying on hash equality alone.

AURA-HOI separates three views:

1. **Raw view:** emit every itemset satisfying the selected threshold predicate.
2. **Closed view:** emit one support-closed representative per support-equivalence class.
3. **Top- k view:** maintain an optional bounded heap over the same scoring evidence.

The present experiments use the raw view only, because HEP and DFHOI are evaluated as raw fullset baselines.

Running example: raw and closed views. Using D_{ex} , consider the itemset $X = \{A, C\}$. Its support set is

$$T(\{A, C\}) = \{1, 2, 4\}.$$

The support closure is

$$\text{cl}(\{A, C\}) = t_1 \cap t_2 \cap t_4 = \{A, B, C, D\} \cap \{A, B, C\} \cap \{A, C, D, E\} = \{A, C\}.$$

Thus $\{A, C\}$ is already support-closed.

Now consider $X = \{A, B, C\}$. It appears in t_1 and t_2 , so

$$T(\{A, B, C\}) = \{1, 2\}.$$

Its closure is

$$\text{cl}(\{A, B, C\}) = t_1 \cap t_2 = \{A, B, C, D\} \cap \{A, B, C\} = \{A, B, C\}.$$

Therefore, it is also closed in this example. In raw mode, AURA-HOI would emit an itemset whenever the selected threshold predicate is satisfied. In closed mode, AURA-HOI stores one support-class representative in the ledger, together with its support bitset. If a second generator reaches the same support bitset, the ledger detects the duplicate class by hash lookup followed by byte-level bitset verification.

For $\alpha_{\text{ex}} = 0.35$, the itemset $\{A, B\}$ is accepted in raw summed-compatible mode as shown above. The itemset $\{A, B, C\}$ has

$$\text{supp}(\{A, B, C\}) = 2,$$

and

$$\text{socc}(\{A, B, C\}) = 3 \left(\frac{1}{4} + \frac{1}{3} \right) = 1.75.$$

Since $1.75 \geq \xi = 1.75$, $\{A, B, C\}$ is also accepted under the compatibility predicate. Thus, the running example gives at least two concrete raw accepted patterns under the shared threshold.

3.4 Vertical bitset evidence

For each item $i \in I$, AURA-HOI builds a vertical bitset B_i over transaction identifiers. For a prefix P , the support bitset is

$$B_P = \bigcap_{i \in P} B_i. \quad (8)$$

The population count of B_P gives $\text{supp}(P)$. Exact occupancy is computed using a reciprocal transaction-length array $r_q = 1/|t_q|$. This representation makes support evidence explicit: support counting, equality testing, closure checking, and audit certificates are all derived from the same bitset object.

Running example: vertical bitsets. For D_{ex} , define bit positions in the order $(t_1, t_2, t_3, t_4, t_5)$. The vertical bitsets are

$$B_A = 11110, \quad B_B = 11101, \quad B_C = 11011, \quad B_D = 10111, \quad B_E = 00011.$$

Here, a 1 means that the item appears in the corresponding transaction. For example,

$$B_A = 11110$$

means that A appears in t_1, t_2, t_3, t_4 but not in t_5 .

For the prefix $P = \{A, B\}$, AURA-HOI computes the support bitset by intersection:

$$B_{\{A,B\}} = B_A \cap B_B = 11110 \cap 11101 = 11100.$$

Thus

$$T(\{A, B\}) = \{1, 2, 3\}, \quad \text{supp}(\{A, B\}) = 3,$$

which matches the direct transaction-level calculation. This is the same evidence used for support pruning, occupancy computation, closure checking, and ledger certification.

For $P = \{A, B, C\}$,

$$B_{\{A,B,C\}} = B_A \cap B_B \cap B_C = 11110 \cap 11101 \cap 11011 = 11000.$$

Thus $T(\{A, B, C\}) = \{1, 2\}$ and $\text{supp}(\{A, B, C\}) = 2$.

3.5 Occupancy envelope

Let P be a prefix, let $S = T(P)$ be its support set, let $p = |P|$, and let R be the remaining extension tail. For a descendant $Y = P \cup Z$ with $Z \subseteq R$, the support set satisfies $T(Y) \subseteq S$. Let $r_{S,\ell}^\downarrow$ be the ℓ th largest reciprocal transaction length among transactions in S . A safe average-occupancy envelope is

$$\text{Env}(P, R) = \max_{1 \leq u \leq |S|} \max_{0 \leq b \leq |R|} \frac{p+b}{u} \sum_{\ell=1}^u r_{S,\ell}^\downarrow. \quad (9)$$

The bound is clipped at 1 in average mode.

Lemma 1 (Envelope safety). *For every descendant $Y = P \cup Z$ with $Z \subseteq R$, $\text{aocc}(Y) \leq \text{Env}(P, R)$.*

Proof. Let $u = \text{supp}(Y)$ and $b = |Z|$. Because $T(Y) \subseteq S$, the sum of reciprocal transaction lengths over $T(Y)$ is no larger than the sum of the u largest reciprocal lengths in S . Also, $|Y| = p + b$ for some $0 \leq b \leq |R|$. Substituting these relaxations into the average-occupancy definition yields a term inside the maximization defining $\text{Env}(P, R)$. Therefore $\text{aocc}(Y) \leq \text{Env}(P, R)$. $\square$

Running example: envelope intuition. Let the current prefix be $P = \{A\}$ and the remaining tail be

$$R = \{B, C, D, E\}.$$

The support set of P is

$$T(\{A\}) = \{1, 2, 3, 4\},$$

because A appears in the first four transactions. The reciprocal transaction lengths over this support are

$$\left\{ \frac{1}{4}, \frac{1}{3}, \frac{1}{3}, \frac{1}{4} \right\}.$$

Sorted in descending order, this becomes

$$r_S^\downarrow = \left\{ \frac{1}{3}, \frac{1}{3}, \frac{1}{4}, \frac{1}{4} \right\}.$$

The envelope asks: among all possible descendants of $\{A\}$, what is the largest average occupancy that could still be achieved if the best supporting transactions and the largest feasible extension length were chosen? This value is an upper bound, not necessarily the score of a real itemset. Therefore, if the envelope is already below the threshold, every descendant can be safely pruned.

For example, if a descendant keeps $u = 2$ supporting transactions and extends P by $b = 2$ items, the relaxed occupancy term is

$$\frac{1+2}{2} \left(\frac{1}{3} + \frac{1}{3} \right) = 1.0.$$

After clipping in average mode, this candidate upper bound is 1.0. Since this is only an upper bound, it may be loose, which motivates the residual envelope.

3.6 Residual occupancy-envelope pruning

The previous envelope can be loose because it assumes tail items can co-occur in every supporting transaction. AURA-HOI refines it with residual co-occurrence capacity. For each $q \in S$, define

$$c_R(q) = |t_q \cap R|. \quad (10)$$

A descendant can add at most $c_R(q)$ tail items while remaining inside t_q . The residual envelope is

$$\text{Env}_R(P, R) = \max_{1 \leq u \leq |S|} \frac{1}{u} \sum_{\ell=1}^u \frac{p + c_R(q_\ell)}{|t_{q_\ell}|}, \quad (11)$$

where $q_1, \dots, q_u$ are selected as the transactions with the largest residual occupancy contributions.

Theorem 1 (No false-negative pruning). *If AURA-HOI prunes a node P only when $\text{supp}(P) < \sigma_{\min}$ or $\min\{\text{Env}(P, R), \text{Env}_R(P, R)\} < \alpha$, then no valid high-occupancy descendant of P is removed.*

Proof. The support condition is safe by support anti-monotonicity: every descendant of P has support at most $\text{supp}(P)$. The first occupancy condition is safe by the envelope lemma. For the residual envelope, a descendant can include no more than $p + c_R(q)$ items inside transaction $q \in T(P)$, so its per-transaction occupancy contribution is bounded by $(p + c_R(q))/|t_q|$. Maximizing over the best possible u supporting transactions yields $\text{Env}_R(P, R)$. Hence every descendant has occupancy at most $\min\{\text{Env}(P, R), \text{Env}_R(P, R)\}$. If this value is below α , no descendant can satisfy the threshold. $\square$

Running example: residual capacity. Again let $P = \{A\}$ and $R = \{B, C, D, E\}$. For each transaction supporting P , the residual capacity $c_R(q)$ counts how many tail items still occur in that transaction:

q	t_q	$c_R(q)$
1	$\{A, B, C, D\}$	3
2	$\{A, B, C\}$	2
3	$\{A, B, D\}$	2
4	$\{A, C, D, E\}$	3

Thus the residual per-transaction upper contributions are

$$\frac{1+3}{4} = 1.0, \quad \frac{1+2}{3} = 1.0, \quad \frac{1+2}{3} = 1.0, \quad \frac{1+3}{4} = 1.0.$$

For this prefix, the residual envelope remains high because every supporting transaction still contains several tail items. Therefore, AURA-HOI should not prune the branch rooted at $\{A\}$ too early.

Now consider a deeper prefix $P = \{A, B, C\}$. Its support is

$$T(\{A, B, C\}) = \{1, 2\}.$$

If the tail after this prefix is $R = \{D, E\}$, then

$$c_R(1) = 1, \quad c_R(2) = 0,$$

because t_1 contains D but t_2 contains neither D nor E . The residual upper contributions are

$$\frac{3+1}{4} = 1.0, \quad \frac{3+0}{3} = 1.0.$$

This branch may still be promising in average mode. However, for prefixes whose supporting transactions have little remaining tail capacity, Env_R becomes much smaller and can safely prune the subtree. This is the purpose of residual occupancy-envelope pruning: it rules out descendants that cannot reach the threshold even under the best possible extension.

3.7 Summed-compatible upper bound in raw mode

In the C implementation, raw summed-compatible mode uses a specialized path. Instead of computing closure and ledger entries for every accepted itemset, the raw path enumerates fullset itemsets and records raw counts. For a support bitset B_P , the implementation computes the reciprocal sum

$$\sum_{q \in T(P)} \frac{1}{|t_q|}$$

and uses it to evaluate

$$\text{socc}(P) = |P| \sum_{q \in T(P)} \frac{1}{|t_q|}.$$

The implementation also computes a summed upper bound over the current support by grouping transactions by length. This corresponds to the purpose of `support_summed_stats` in the C implementation: the same support bitset provides both the exact reciprocal sum and an upper-bound value used for pruning.

Running example: summed raw path. For $P = \{A, B\}$, the support bitset is 11100, so the reciprocal sum is

$$\frac{1}{4} + \frac{1}{3} + \frac{1}{3} = 0.9167.$$

The summed score is

$$\text{socc}(\{A, B\}) = 2 \times 0.9167 = 1.8333.$$

Since $\xi = 1.75$ when $\alpha_{\text{ex}} = 0.35$, AURA-HOI accepts this itemset in raw summed-compatible mode. In the implementation, raw mode increments the raw HOI count and raw output-size statistics, rather than inserting the itemset into the closed ledger.

Algorithm 1 AURA-HOI search with raw and closed output views

Require: Transaction database D , threshold α , output view $v \in \{\text{raw}, \text{closed}\}$

Ensure: Raw HOIs or support-closed representative ledger

```
1: Build vertical bitsets  $B_i$  and reciprocal transaction lengths  $r_q = 1/|t_q|$ 
2: Set  $\sigma_{\min} \leftarrow \lceil \alpha |D| \rceil$  in summed-compatibility mode
3: Initialize ledger  $L$  and optional top- $k$  heap  $Q_k$ 
4: procedure SEARCH( $P, B_P, R$ )
5:   if popcnt( $B_P$ ) <  $\sigma_{\min}$  then
6:     return
7:   end if
8:   if min{Env( $P, R$ ), Env $_R(P, R)$ } <  $\alpha$  in average mode then
9:     return
10:  end if
11:  if  $v = \text{closed}$  then
12:     $C \leftarrow \text{cl}(P)$  using support-subset tests over vertical bitsets
13:  else
14:     $C \leftarrow P$ 
15:  end if
16:  Compute selected occupancy score of  $C$  from  $B_P$  and  $r_q$ 
17:  if  $C$  satisfies the selected threshold predicate then
18:    if  $v = \text{raw}$  then
19:      Emit  $C$ 
20:    else
21:      Insert  $(C, B_P, \text{aocc}(C), \text{supp}(C))$  into ledger  $L$  if the support class is new
22:    end if
23:  end if
24:   $R' \leftarrow \{i \in R : i > \max(C), i \notin C\}$ 
25:  for all  $i \in R'$  in canonical order do
26:     $B' \leftarrow B_P \cap B_i$ 
27:    SEARCH( $C \cup \{i\}, B', \{j \in R' : j > i\}$ )
28:  end for
29: end procedure
30: SEARCH( $\emptyset, \{1, \dots, n\}, I$ )
31: return emitted raw itemsets or ledger  $L$ 
```

3.8 Algorithm

Algorithm 1 gives the high-level search procedure. In raw mode, the closure jump is disabled and the algorithm emits the full raw set. In closed mode, the algorithm computes support closure and inserts a representative into the support-class ledger.

Running example: raw search trace. Assume summed-occupancy compatibility with $\alpha_{\text{ex}} = 0.35$ on D_{ex} . Then

$$\sigma_{\min} = 2, \quad \xi = 1.75.$$

Suppose the search starts from the empty prefix and considers item A . The singleton support is

$$B_A = 11110, \quad \text{supp}(\{A\}) = 4.$$

The summed occupancy is

$$\text{socc}(\{A\}) = 1 \left(\frac{1}{4} + \frac{1}{3} + \frac{1}{3} + \frac{1}{4} \right) = 1.1667.$$

Although $\{A\}$ passes support, it does not pass summed occupancy because $1.1667 < 1.75$. However, it cannot be discarded solely for failing occupancy, because descendants may have larger itemset length and thus larger summed occupancy.

The search then considers $\{A, B\}$:

$$B_{\{A,B\}} = 11100, \quad \text{supp}(\{A, B\}) = 3,$$

and

$$\text{socc}(\{A, B\}) = 2 \left(\frac{1}{4} + \frac{1}{3} + \frac{1}{3} \right) = 1.8333.$$

Since $1.8333 \geq 1.75$, AURA-HOI emits $\{A, B\}$ in raw summed-compatible mode.

Next consider $\{A, B, C\}$:

$$B_{\{A,B,C\}} = 11000, \quad \text{supp}(\{A, B, C\}) = 2.$$

Its summed occupancy is

$$\text{socc}(\{A, B, C\}) = 3 \left(\frac{1}{4} + \frac{1}{3} \right) = 1.75.$$

Since $1.75 \geq 1.75$, AURA-HOI also emits $\{A, B, C\}$.

Now consider $\{A, B, D\}$:

$$B_{\{A,B,D\}} = 10100, \quad \text{supp}(\{A, B, D\}) = 2,$$

and

$$\text{socc}(\{A, B, D\}) = 3 \left(\frac{1}{4} + \frac{1}{3} \right) = 1.75.$$

This itemset is also accepted. These steps demonstrate the behavior of the raw fast path: it repeatedly intersects support bitsets, applies support pruning, computes summed occupancy from reciprocal transaction lengths, and emits accepted raw itemsets without inserting them into the closed ledger.

Running example: closed search trace. If the same branch is evaluated in closed mode, AURA-HOI additionally computes support closure. For example, after reaching $\{A, B\}$ with support set $\{1, 2, 3\}$, the closure is

$$\text{cl}(\{A, B\}) = t_1 \cap t_2 \cap t_3 = \{A, B, C, D\} \cap \{A, B, C\} \cap \{A, B, D\} = \{A, B\}.$$

Thus $\{A, B\}$ is its own closed representative. The ledger stores the support bitset 11100, the representative $\{A, B\}$, the support count 3, and the occupancy score. If another generator reached the same support bitset, the ledger would identify it as a duplicate support class.

3.9 Complexity

Let $w = \lceil n/64 \rceil$ be the number of machine words per bitset and let N_{vis} be the number of visited search nodes after pruning. Each vertical intersection costs $O(w)$ word operations. A direct closure test costs $O(|R|w)$. Occupancy computation costs $O(w + \text{supp}(X))$ when scanning set bits. Therefore, the running time is

$$T_{\text{AURA}} = O \left(N_{\text{vis}}w + \sum_{C \in L} (w + \text{supp}(C)) + T_{\text{env}} \right), \quad (12)$$

where T_{env} is the cost of computing occupancy envelopes. The main memory cost is

$$M_{\text{AURA}} = O(mw) + O(d_{\text{max}}w) + O(n) + O(|L|), \quad (13)$$

where mw stores item bitsets, $d_{\text{max}}w$ stores recursion temporaries, $O(n)$ stores transaction-length evidence, and $O(|L|)$ stores support-class certificates.

Running example: cost interpretation. For D_{ex} , there are only $n = 5$ transactions, so each support bitset fits into one 64-bit machine word:

$$w = \left\lceil \frac{5}{64} \right\rceil = 1.$$

Therefore, each bitset intersection such as

$$B_{\{A,B\}} = B_A \cap B_B$$

costs one machine-word operation in the idealized model. On real benchmark datasets such as **kosarak**, n is much larger and w grows accordingly. This explains why vertical bitset methods are very efficient on dense datasets but can pay more memory and scanning cost on large sparse datasets. The running example shows the same operations in miniature: support is always derived from bitset intersections, while occupancy is derived from scanning the set bits and summing reciprocal transaction lengths.

4 Experiment Results

4.1 Experimental environment

The implementation is written in C and compiled as part of the **dm** executable. All experiments were executed on a laptop-scale machine: HP ProBook 450 G5, Intel Core i7-8550U processor, and 16GB RAM. The compared algorithms are **hep**, **dfhoi**, and **aura_hoi**. All algorithms are executed on the same dataset files and the same threshold grid. AURA-HOI is run in **sum raw** mode to match the raw summed-occupancy behavior of HEP and DFHOI.

4.2 Datasets and thresholds

Table 2 gives the benchmark matrix. The shared threshold α controls both support and summed occupancy according to Eq. (5). Therefore, for a dataset of size n , each threshold corresponds to $\sigma_{\min} = \lceil \alpha n \rceil$.

Table 2: Datasets and threshold grids used in the final benchmark.

Dataset	Transactions	Profile	Shared thresholds α	MinSup range
mushrooms	8,416	dense categorical benchmark	0.05; 0.075; 0.1; 0.125; 0.15; 0.2; 0.3; 0.4	421–3367
chess	3,196	dense benchmark	0.5; 0.55; 0.6; 0.65; 0.7; 0.75; 0.8; 0.85	1598–2717
retail	88,162	sparse retail transactions	0.005; 0.01; 0.02; 0.03; 0.05; 0.075; 0.1	441–8817
T10I4D100K	100,000	synthetic sparse transactions	0.005; 0.01; 0.02; 0.03; 0.05; 0.075; 0.1	500–10000
kosarak	990,002	large sparse clickstream	0.001; 0.002; 0.005; 0.01; 0.02	991–19801

4.3 Correctness protocol

For each dataset and threshold, all algorithms must emit the same raw itemset count before runtime and memory comparisons are interpreted. Let R_A and R_B be normalized outputs from algorithms A and B . The count difference and Jaccard similarity are

$$\text{CountDiff}(A, B) = ||R_A| - |R_B||, \quad \text{Jaccard}(A, B) = \frac{|R_A \cap R_B|}{|R_A \cup R_B|}. \quad (14)$$

The current final benchmark records itemset counts for every dataset–threshold configuration. Table 3 shows that the three algorithms match on all reported configurations.

Table 3: Raw-output equivalence under the shared support/occupancy threshold.

Dataset	α	$\sigma_{\min}$	HEP	DFHOI	AURA-HOI	Match
T10I4D100K	0.005	500	10	10	10	yes
T10I4D100K	0.01	1000	0	0	0	yes
T10I4D100K	0.02	2000	0	0	0	yes
T10I4D100K	0.03	3000	0	0	0	yes
T10I4D100K	0.05	5000	0	0	0	yes
T10I4D100K	0.075	7500	0	0	0	yes
T10I4D100K	0.1	10000	0	0	0	yes
chess	0.5	1598	0	0	0	yes
chess	0.55	1758	0	0	0	yes
chess	0.6	1918	0	0	0	yes
chess	0.65	2078	0	0	0	yes
chess	0.7	2238	0	0	0	yes
chess	0.75	2397	0	0	0	yes
chess	0.8	2557	0	0	0	yes
chess	0.85	2717	0	0	0	yes
kosarak	0.001	991	726	726	726	yes
kosarak	0.002	1981	265	265	265	yes
kosarak	0.005	4951	86	86	86	yes
kosarak	0.01	9901	31	31	31	yes
kosarak	0.02	19801	14	14	14	yes
mushrooms	0.05	421	93266	93266	93266	yes
mushrooms	0.075	632	16613	16613	16613	yes
mushrooms	0.1	842	955	955	955	yes
mushrooms	0.125	1052	12	12	12	yes
mushrooms	0.15	1263	2	2	2	yes
mushrooms	0.2	1684	0	0	0	yes
mushrooms	0.3	2525	0	0	0	yes
mushrooms	0.4	3367	0	0	0	yes
retail	0.005	441	37	37	37	yes
retail	0.01	882	15	15	15	yes
retail	0.02	1764	9	9	9	yes
retail	0.03	2645	3	3	3	yes
retail	0.05	4409	3	3	3	yes
retail	0.075	6613	2	2	2	yes
retail	0.1	8817	0	0	0	yes

4.4 Performance overview

Table 4 summarizes the runtime and memory comparison. A speed ratio above 1 in the HEP/AURA or DFHOI/AURA columns means that AURA-HOI is faster on the median threshold of that dataset. The results show that AURA-HOI is consistently faster than HEP on **chess**, **retail**, and **T10I4D100K**; competitive on high-threshold **kosarak**; and slower on the lowest-threshold, high-output **mushrooms** and **kosarak** settings. This behavior is expected for an audit-oriented vertical-bitset implementation: low thresholds increase output volume and expose overheads, whereas moderate and high thresholds allow the pruning and raw fast path to dominate.

Table 4: Aggregate runtime and memory summary. Ratios are median speed ratios over the dataset’s threshold grid.

Dataset	N_α	AURA < HEP time	AURA < DFHOI time	HEP/AURA	DFHOI/AURA	AURA < HEP RAM	AURA < DFHOI RAM
mushrooms	8	2/8	0/8	0.86×	0.47×	8/8	8/8
chess	8	8/8	0/8	1.57×	0.93×	8/8	8/8
retail	7	6/7	6/7	1.24×	1.23×	6/7	5/7
T10I4D100K	7	7/7	7/7	1.31×	1.26×	4/7	4/7
kosarak	5	2/5	2/5	0.55×	0.55×	2/5	2/5

Table 5 reports representative raw measurements from the CSV file. The full result file is part of the reproducibility artifact.

Table 5: Representative runtime and memory measurements from the final benchmark.

Dataset	α	Algorithm	Time (s)	Peak RAM (MB)	Itemsets
mushrooms	0.05	HEP	13.196	3823.48	93266
mushrooms	0.05	DFHOI	6.227	303.45	93266
mushrooms	0.05	AURA-HOI	21.926	302.08	93266
mushrooms	0.15	HEP	0.728	525.5	2
mushrooms	0.15	DFHOI	0.423	303.02	2
mushrooms	0.15	AURA-HOI	0.834	302.06	2
mushrooms	0.4	HEP	0.019	306.09	0
mushrooms	0.4	DFHOI	0.015	301.94	0
mushrooms	0.4	AURA-HOI	0.016	301.73	0
chess	0.5	HEP	13.099	3915.96	0
chess	0.5	DFHOI	8.081	302.87	0
chess	0.5	AURA-HOI	8.233	301.39	0
chess	0.7	HEP	0.587	508.42	0
chess	0.7	DFHOI	0.406	302.16	0
chess	0.7	AURA-HOI	0.406	301.55	0
chess	0.85	HEP	0.044	317.66	0
chess	0.85	DFHOI	0.027	301.82	0
chess	0.85	AURA-HOI	0.028	301.56	0
retail	0.005	HEP	0.106	310.02	37
retail	0.005	DFHOI	0.101	310.09	37
retail	0.005	AURA-HOI	0.16	311.14	37
retail	0.03	HEP	0.067	309.62	3
retail	0.03	DFHOI	0.066	309.62	3
retail	0.03	AURA-HOI	0.053	308.93	3
retail	0.1	HEP	0.065	309.3	0
retail	0.1	DFHOI	0.072	309.41	0
retail	0.1	AURA-HOI	0.053	308.89	0
T10I4D100K	0.005	HEP	1.625	312.92	10
T10I4D100K	0.005	DFHOI	1.573	312.92	10
T10I4D100K	0.005	AURA-HOI	0.733	316.31	10
T10I4D100K	0.03	HEP	0.075	310.24	0
T10I4D100K	0.03	DFHOI	0.073	310.29	0
T10I4D100K	0.03	AURA-HOI	0.061	309.93	0
T10I4D100K	0.1	HEP	0.065	309.75	0
T10I4D100K	0.1	DFHOI	0.066	309.63	0
T10I4D100K	0.1	AURA-HOI	0.053	309.51	0
kosarak	0.001	HEP	9.561	459.96	726
kosarak	0.001	DFHOI	9.581	423.77	726

Dataset	α	Algorithm	Time (s)	Peak RAM (MB)	Itemsets
kosarak	0.001	AURA-HOI	277.798	546.12	726
kosarak	0.005	HEP	0.952	411.36	86
kosarak	0.005	DFHOI	0.956	410.46	86
kosarak	0.005	AURA-HOI	1.737	415.36	86
kosarak	0.02	HEP	0.627	406.4	14
kosarak	0.02	DFHOI	0.628	406.51	14
kosarak	0.02	AURA-HOI	0.486	399.89	14

4.5 Dataset-level charts

Figures 1–5 show the final line charts. Each chart reports runtime and peak resident memory across thresholds for one dataset. The x-axis is the shared support/occupancy threshold α , not a standalone occupancy-only threshold.

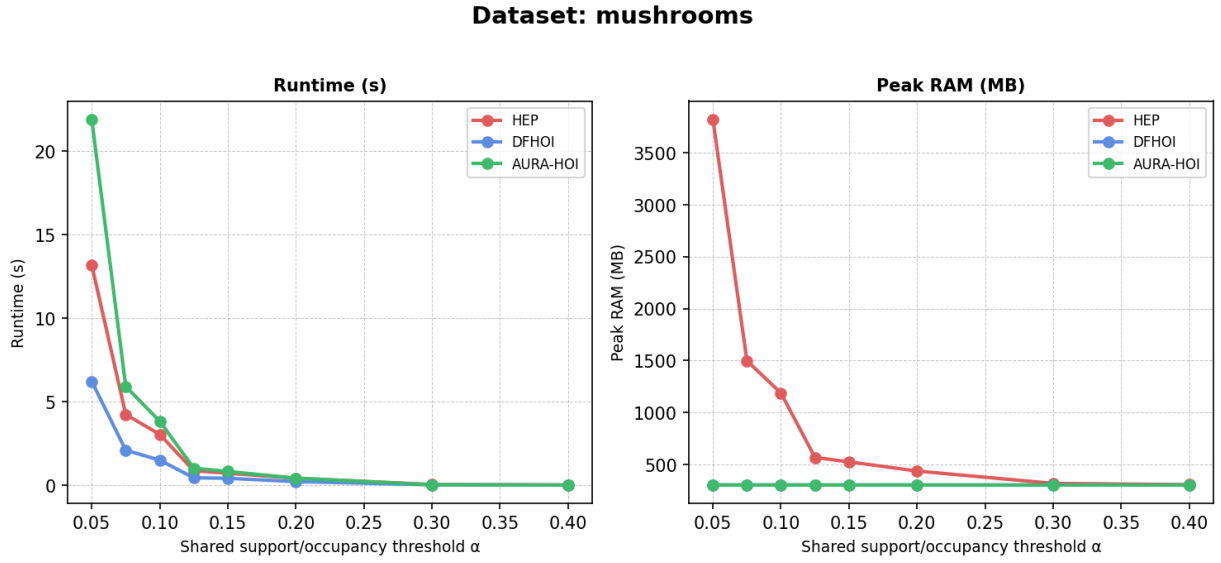


Figure 1: Runtime and peak RAM on *mushrooms*. The low-threshold region produces very large raw outputs; all algorithms emit identical counts, but AURA-HOI pays additional audit and bitset overhead at the lowest thresholds.

Dataset: chess

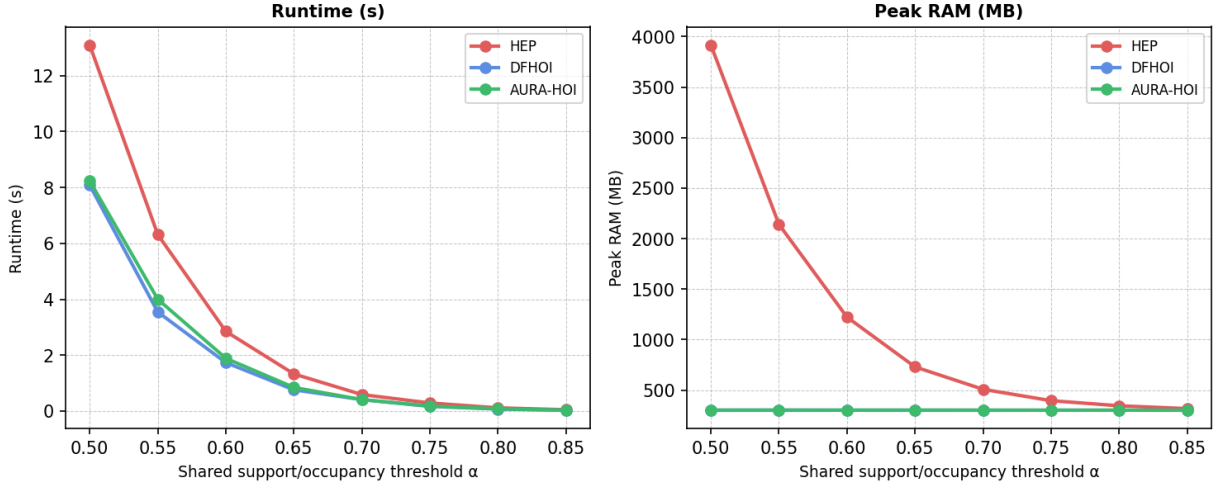


Figure 2: Runtime and peak RAM on **chess**. AURA-HOI is faster than HEP across the threshold grid while using stable memory. DFHOI remains slightly faster in runtime, but with similar memory to AURA-HOI.

Dataset: retail

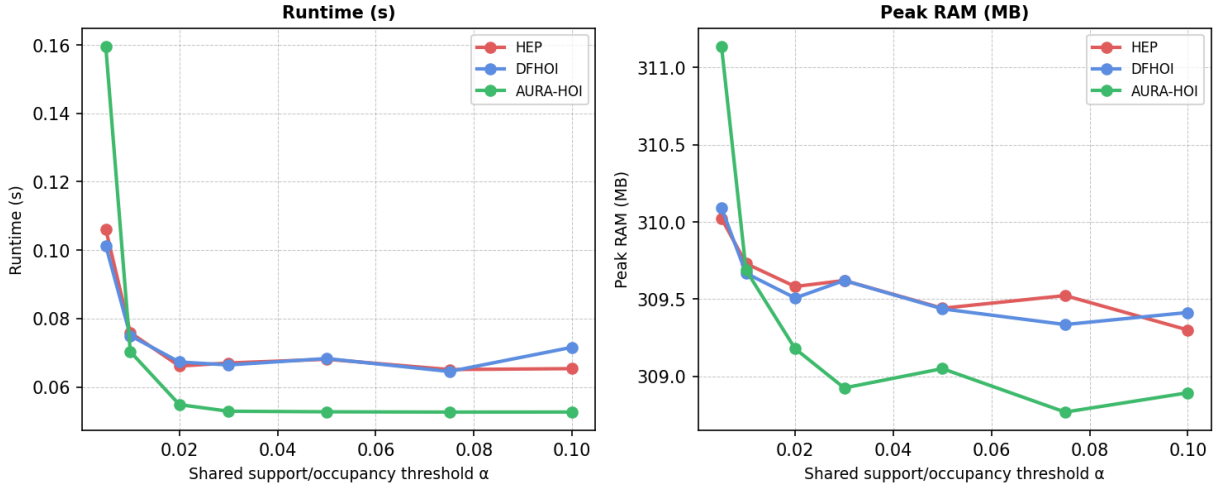


Figure 3: Runtime and peak RAM on **retail**. The sparse workload has very small output under the shared threshold. AURA-HOI becomes fastest after the lowest threshold and uses slightly lower resident memory than the baselines.

Dataset: T10I4D100K

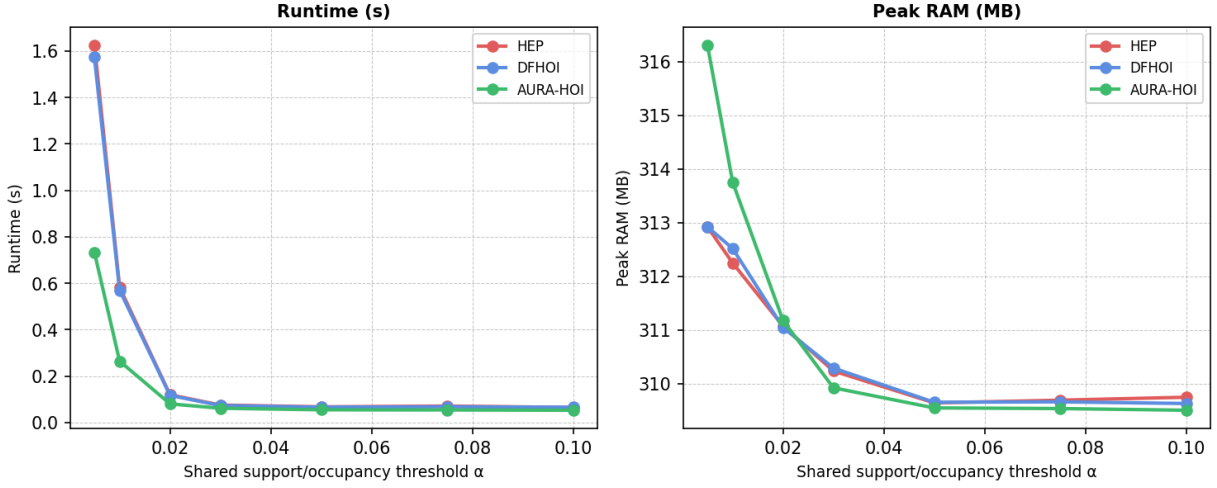


Figure 4: Runtime and peak RAM on T10I4D100K. AURA-HOI is faster than both baselines at every tested threshold, and the memory curves converge once the threshold removes almost all output.

Dataset: kosarak

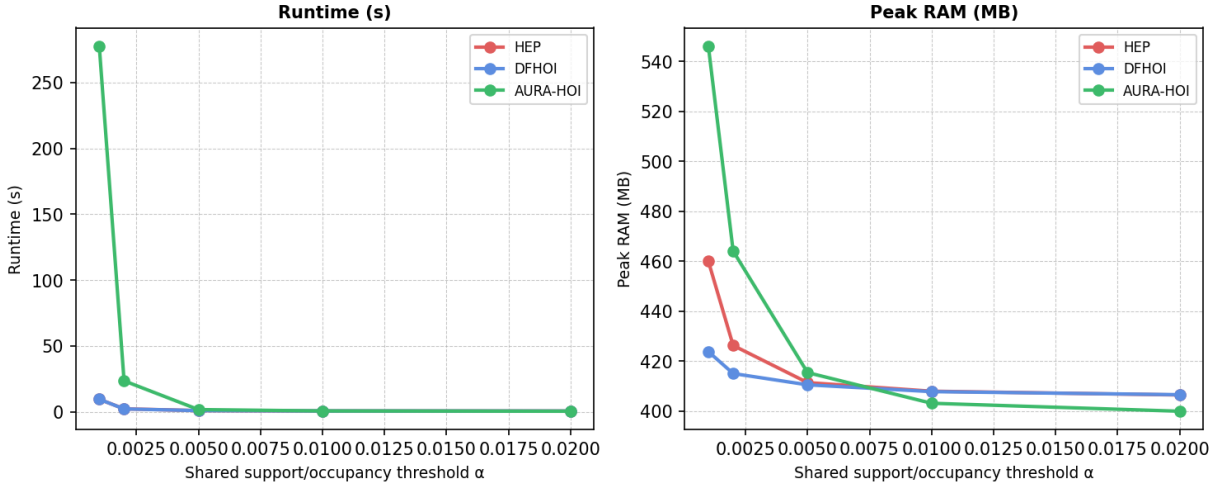


Figure 5: Runtime and peak RAM on kosarak. The lowest threshold is the most difficult setting for AURA-HOI, while moderate and high thresholds show competitive or better runtime and memory. This dataset highlights the importance of reporting threshold sensitivity rather than a single aggregate score.

4.6 Interpretation

The most important result is not that one algorithm dominates every setting. Rather, the benchmark shows that all runtime and memory comparisons are made under matched raw-output semantics. This matters because the itemset counts match for every one of the 35 dataset–threshold configurations. On dense **chess**, AURA-HOI is consistently faster than HEP and uses far less memory than HEP at low thresholds. On **retail** and T10I4D100K, AURA-HOI is fastest for most or all thresholds, although the absolute runtimes are small. On **mushrooms** and **kosarak**, AURA-HOI is slower at the lowest thresholds, where the output is larger and the overhead of the audit-oriented bitset design is most visible.

These results support a conservative claim: AURA-HOI is an exact raw-HOI miner with auditable support evidence and competitive performance. It is not presented as universally faster than DFHOI on every dataset. Instead, its contribution is the combination of raw-output compatibility, support-class auditability, and stable memory behavior.

5 Discussion

5.1 Why shared-threshold compatibility matters

The final experiments use a shared support/occupancy threshold because the baseline implementations treat the command-line threshold as both a support threshold and a summed-occupancy threshold. If AURA-HOI used an independently chosen support threshold, then the compared algorithms might solve different mining tasks. Eq. (5) prevents this problem by forcing all algorithms to use the same raw predicate.

5.2 What the running example shows

The running example clarifies four design decisions. First, support and occupancy are different signals. In D_{ex} , $\{A, B\}$ has support 3 and average occupancy 0.6111, while its summed-compatible score is 1.8333. Second, the same itemset may pass or fail depending on whether the average predicate or summed-compatible predicate is used. This is why the paper explicitly separates mathematical HOI semantics from repository compatibility. Third, raw and closed outputs are different views. A raw miner asks whether each itemset satisfies the threshold, whereas a closed-view miner stores representatives of support-equivalence classes. Fourth, AURA-HOI uses one support-evidence object for multiple purposes: the same bitset intersection gives support, enables closure tests, supports ledger equality, and provides the transaction identifiers needed for exact occupancy computation. This is why the algorithm is described as auditable rather than only as a faster enumerator.

5.3 Runtime trade-offs

The charts show that AURA-HOI is not uniformly faster at the lowest thresholds. This is a useful result rather than a failure. Low thresholds create large raw outputs and expose the cost of maintaining audit-ready support evidence. In contrast, moderate and high thresholds often allow AURA-HOI to benefit from pruning, raw-mode traversal, and stable memory behavior. A future hybrid version could switch between sparse list-based projection and dense bitset traversal according to dataset density.

5.4 Memory behavior

HEP shows large memory spikes on dense datasets, especially `mushrooms` and `chess`. AURA-HOI has much flatter memory behavior because its main representation is vertical bitset evidence plus recursion temporaries. On sparse datasets, peak resident memory is close across all algorithms, suggesting that dataset loading and runtime environment overhead dominate the algorithm-specific memory difference.

5.5 Threats to validity

There are several threats to validity. First, the results are measured on a laptop CPU rather than a controlled server. This is acceptable for reproducibility but should not be interpreted as definitive hardware-independent performance. Second, the experiments use one implementation repository; implementation differences may affect absolute runtime. Third, itemset-count equality is necessary but weaker than full normalized itemset equality. A final archival artifact should include a checker that canonicalizes itemsets and verifies Jaccard similarity, not only counts. Fourth, disk footprint is

not reported in the final CSV used for this paper, so the current final version focuses on runtime, peak RAM, and itemset counts rather than disk size.

5.6 Limitations

AURA-HOI is currently an HOI-only miner. It does not handle utility, profit, sequence, temporal, or shelf-time information. Its closed-ledger view is designed for compact analysis but should not replace raw output when the user needs every threshold-satisfying itemset. The final benchmark also excludes `pumsb` because it exceeded the available 16GB memory budget under attempted laptop-scale configurations. This exclusion is reported explicitly to avoid presenting an incomplete or unstable benchmark as final.

5.7 Reproducibility

The reproducibility package should include the C source code, dataset paths, benchmark script, raw CSV files, itemset-count CSV, generated figures, and the exact $\text{\LaTeX}$ source. Each result row should record dataset, algorithm, shared threshold α , computed $\sigma_{\min}$, runtime, peak resident memory, and itemset count. Pattern contents should be emitted only when correctness checking is explicitly requested; normal benchmark runs should output statistics by default.

6 Conclusion

This paper presented AURA-HOI, an auditable high-occupancy itemset mining algorithm that separates score semantics from output views. The method combines vertical bitset evidence, residual occupancy-envelope pruning, and support-class certificates. A five-transaction running example demonstrates how support, average occupancy, summed compatibility, closure, bitset evidence, and raw search operate from the same data. The final benchmark compares AURA-HOI against HEP and DFHOI under a shared support/occupancy threshold on five datasets. Across all 35 reported configurations, the algorithms emit identical raw itemset counts, establishing matched output semantics before performance interpretation. AURA-HOI is faster than HEP on most configurations, competitive with DFHOI overall, and generally exhibits stable peak memory behavior. The main value of AURA-HOI is therefore not a universal speed claim, but a scientifically auditable HOI mining framework in which raw, closed, and representative outputs can be traced back to common support evidence.

References

- [1] Rakesh Agrawal, Tomasz Imieliński, and Arun Swami. Mining association rules between sets of items in large databases. In *Proceedings of the 1993 ACM SIGMOD International Conference on Management of Data*, pages 207–216, 1993. doi:10.1145/170035.170072.
- [2] Rakesh Agrawal and Ramakrishnan Srikant. Fast algorithms for mining association rules in large databases. In *Proceedings of the 20th International Conference on Very Large Data Bases*, pages 487–499, 1994.
- [3] Zhi-Hong Deng. Mining high occupancy itemsets. *Future Generation Computer Systems*, 102:222–229, 2020. doi:10.1016/j.future.2019.07.039.
- [4] Jiawei Han, Jian Pei, and Yiyen Yin. Mining frequent patterns without candidate generation. In *Proceedings of the 2000 ACM SIGMOD International Conference on Management of Data*, pages 1–12, 2000. doi:10.1145/335191.335372.
- [5] Hyojun Kim, Taehyung Ryu, Changhoon Lee, Hyunmin Kim, Truong Truong, Philippe Fournier-Viger, Witold Pedrycz, and Unil Yun. Mining high occupancy patterns to analyze incremental data in intelligent systems. *ISA Transactions*, 131:460–475, 2022.

- [6] José María Luna, Ramesh Uday Kiran, Philippe Fournier-Viger, and Sebastián Ventura. Frequent itemset mining: A 25 years review. *Wiley Interdisciplinary Reviews: Data Mining and Knowledge Discovery*, 9(6):e1329, 2019. doi:10.1002/widm.1329.
- [7] Loan T. T. Nguyen, Thang Mai, Giao-Huy Pham, Unil Yun, and Bay Vo. An efficient method for mining high occupancy itemsets based on equivalence class and early pruning. *Knowledge-Based Systems*, 267:110441, 2023. doi:10.1016/j.knosys.2023.110441.
- [8] Nicolas Pasquier, Yves Bastide, Rafik Taouil, and Lotfi Lakhal. Discovering frequent closed itemsets for association rules. In *Proceedings of the 7th International Conference on Database Theory*, pages 398–416, 1999. doi:10.1007/3-540-49257-7_25.
- [9] Mohammed J. Zaki, Srinivasan Parthasarathy, Mitsunori Ogihara, and Wei Li. New algorithms for fast discovery of association rules. In *Proceedings of the Third International Conference on Knowledge Discovery and Data Mining*, pages 283–286, 1997.
- [10] Mohammed J. Zaki. Scalable algorithms for association mining. *IEEE Transactions on Knowledge and Data Engineering*, 12(3):372–390, 2000. doi:10.1109/69.846291.
- [11] Mohammed J. Zaki and Ching-Jiu Hsiao. CHARM: An efficient algorithm for closed itemset mining. In *Proceedings of the Second SIAM International Conference on Data Mining Workshop on High Performance Data Mining*, 2002.